

Secure Coding Review

CodeAlpha Cyber Security Internship — Task 3

1. Introduction

Secure coding review is the process of examining an application's source code to identify security vulnerabilities before they can be exploited. It combines manual inspection with static analysis concepts to detect unsafe coding practices.

For this project, a simple Node.js User Management System (login and registration) was selected as the sample application. The application was intentionally written with common, real-world security mistakes for the purpose of this review, and a corrected, secure version was then produced.

2. Objective

The main objectives of this project are:

- To select an application and perform a security-focused code review.
- To identify common security vulnerabilities in the source code.
- To use manual inspection and static-analysis concepts to detect issues.
- To provide recommendations and best practices for secure coding.
- To document findings and suggest concrete remediation steps.

3. Application and Tools Used

Application Reviewed: A simple User Management System (login and registration APIs) built with Node.js and Express.

Tools and methods used for the review:

- Manual Code Review
- ESLint Security Rules (static analysis concept)
- SonarQube (static analysis concept)

4. Vulnerabilities Identified

The following vulnerabilities were identified in `Vulnerable_Code.js`:

Vulnerability	Issue
Hardcoded Password	The admin password is written directly in the source code and visible to anyone with code access.

Vulnerability	Issue
SQL Injection	User input (username/password) is concatenated directly into a SQL query string, allowing query manipulation.
Weak Password Storage	Passwords are stored and compared as plain text instead of being hashed.
Missing Input Validation	Incoming request data (username, password) is used without any validation or sanitization.
Error Information Leakage	Detailed error objects are logged/returned directly to the client, which can expose sensitive internals.

4.1. Hardcoded Password

Vulnerable Code:

```
const adminPassword = "admin123";
```

Issue: The password is embedded directly in the source code, so it is visible to anyone who can read the file or the repository.

Recommended Fix:

- Store secrets in environment variables instead of source code.
- Use a hashed password check rather than a plaintext comparison.

4.2. SQL Injection

Vulnerable Code:

```
const query = "SELECT * FROM users WHERE username='"
+ username +
"' AND password='" + password + "'";
```

Issue: User-supplied input is concatenated directly into the SQL query, allowing an attacker to manipulate the query and potentially bypass authentication or access unauthorized data.

Recommended Fix:

- Use parameterized queries / prepared statements.
- Never build SQL strings via direct concatenation of user input.

4.3. Weak Password Storage

Vulnerable Code:

```
const user = {
  username: username,
  password: password
};
```

Issue: Passwords are stored as plain text, so anyone with database access can read every user's password directly.

Recommended Fix:

- Hash passwords using bcrypt before storing them.
- Never store or compare passwords in plain text.

4.4. Missing Input Validation

Vulnerable Code:

```
const username = req.body.username;
const password = req.body.password;
```

Issue: User input is used immediately without checking whether required fields are present or properly formatted, allowing malformed or malicious input to reach the application logic.

Recommended Fix:

- Validate that required fields exist before processing.
- Sanitize and constrain input formats (e.g., allowed characters, length limits).

4.5. Error Information Leakage

Vulnerable Code:

```
app.use((err, req, res, next) => {
  res.send(err.message);
});
```

Issue: Raw error messages are sent back to the client, which can reveal stack traces, file paths, or other internal details useful to an attacker.

Recommended Fix:

- Log detailed errors on the server only.
- Return generic error messages (e.g., "Internal Server Error") to the client.

5. Secure Code Implementation

Based on the findings above, a secure version of the application (Secure_Code.js) was implemented with the following fixes:

- Removed the hardcoded password from the source code.
- Added password hashing using bcrypt for both registration and login.
- Added input validation to ensure username and password are provided before processing.
- Replaced plain text password storage with securely hashed passwords.
- Hid detailed error messages from the client and logged them securely on the server instead.
- Used bcrypt.compare() for authentication instead of a direct string comparison.

6. Testing

The vulnerable and secure versions were compared by manually reviewing each endpoint's behavior:

- Verified that Vulnerable_Code.js exposed the hardcoded password and stored plain text passwords.
- Verified that Secure_Code.js hashed passwords using bcrypt before storing them.
- Verified that Secure_Code.js rejected registration/login requests missing username or password.
- Verified that Secure_Code.js returned a generic error message instead of raw error details.

7. Result

The code review successfully identified five real-world security vulnerabilities in the sample User Management System: hardcoded credentials, SQL injection, weak password storage, missing input validation, and error

information leakage. Each issue was documented with its risk and a corresponding fix, and a fully corrected secure version of the application was produced.

8. Conclusion

This project demonstrated the importance of secure coding practices in application development. Reviewing code for vulnerabilities such as hardcoded secrets, injection flaws, weak storage, missing validation, and information leakage helps prevent common real-world attacks. Applying fixes like parameterized queries, password hashing, input validation, and proper error handling significantly improves an application's overall security posture.

Author

Nida Kanwal

CodeAlpha Cyber Security Internship